**Khushi Chhaproo | Div.: D20C | Roll No.: 07**

**Blockchain Lab**

## Experiment No 03

**Aim:** To create a cryptocurrency using Python and perform mining in the blockchain created.

### Theory

### 1. Blockchain Overview

Blockchain is a **distributed and decentralized ledger** that stores information in a series of linked blocks. Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash

Once data is recorded, it becomes **immutable** because changing one block requires recalculating all subsequent blocks.

### 2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Solving a computational puzzle using Proof-of-Work.
3. Adding the new block to the blockchain.
4. Broadcasting it to connected peers.

Miners receive cryptocurrency as a reward for successfully mining a block.

**3. Multi-Node Blockchain Network**
In this lab, three independent blockchain nodes are simulated using ports **5001, 5002, and 5003**. Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Connects with peers to share and validate blocks.

**4. Consensus Mechanism**

The **Longest Chain Rule** is used:

- If multiple versions of the chain exist, the longest valid chain is selected.
- This helps all nodes maintain a common transaction history.

**5. Transactions & Mining Reward**

Each transaction contains:

- Sender
- Receiver
- Amount

When a block is mined, pending transactions are added along with a **reward transaction** for the miner.

**6. Chain Replacement**

When `/replace_chain` is called:

1. The node requests chains from its peers.
2. It checks for a longer and valid chain.
3. If found, it replaces its own chain.
4. This keeps the blockchain consistent across nodes.

**Tools & Libraries Used**

- **Python 3.x**
- **Flask** – Web framework for API endpoints
- **Requests** – HTTP communication between nodes
- **Postman** – API testing
- **datetime** – Timestamps
- **hashlib** – SHA-256 hashing
- **uuid4** – Unique identifiers
- **urlparse** – URL processing
- **request** – Handling API requests

**Required Packages**

pip install Flask

pip install requests

**Procedure**

1. Download `hadcoin_node_5001.py`, `hadcoin_node_5002.py`, and `hadcoin_node_5003.py`.
2. Open the files in VS Code.

Install the required packages:

pip install Flask

3. pip install requests
4. Run each node in a separate terminal:

   python hadcoin_node_5001.py

   python hadcoin_node_5002.py

   python hadcoin_node_5003.py
5. Verify that the nodes are running on ports **5001, 5002, and 5003**.
6. Use **Postman** to create transactions, register nodes, mine blocks, view the blockchain,

and test chain replacement.

7. Verify that the blockchain remains valid and synchronized across the nodes.

**OUTPUT:**

**Opened venv and installed python flask**

```
PS C:\Clg\Blockchain codes\Exp 03> python -m venv venv
PS C:\Clg\Blockchain codes\Exp 03> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:\Clg\
Blockchain codes\Exp 03\venv\Scripts\Activate.ps1")
(venv) PS C:\Clg\Blockchain codes\Exp 03> venv\Scripts\activate
(venv) PS C:\Clg\Blockchain codes\Exp 03> pip install Flask
Collecting Flask
  Using cached flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from Flask)
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from Flask)
  Using cached click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from Flask)
  Using cached itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from Flask)
  Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from Flask)
  Using cached markupsafe-3.0.3-cp313-cp313-win_amd64.whl.metadata (2.8 kB)
Collecting werkzeug>=3.1.0 (from Flask)
  Using cached werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Using cached flask-3.1.3-py3-none-any.whl (103 kB)
Using cached blinker-1.9.0-py3-none-any.whl (8.5 kB)
Using cached click-8.5.0-py3-none-any.whl (125 kB)
Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Using cached markupsafe-3.0.3-cp313-cp313-win_amd64.whl (15 kB)
Using cached werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, Flask
Successfully installed Flask-3.1.3 blinker-1.9.0 click-8.5.0 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkz
eug-3.1.8

[notice] A new release of pip is available: 25.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

**hadcoin_node_5001**

```
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> python hardcoin_node_5001.py
 * Serving Flask app 'hardcoin_node_5001'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addreses (0.0.0.0)
 * Running on http://127.0.0.1:5001
 * Running on http://10.124.178.178:5001

127.0.0.1 - - [02/Sep/2026 12:28:37] "POST /connect_node HTTP/1.1" 201 -
127.0.0.1 - - [02/Sep/2026 12:30:36] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [02/Sep/2026 12:40:58] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 - - [02/Sep/2026 12:42:27] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 - - [02/Sep/2026 12:44:28] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [02/Sep/2026 12:44:52] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [02/Sep/2026 12:44:52] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [02/Sep/2026 12:49:32] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [02/Sep/2026 12:51:22] "GET /get_chain HTTP/1.1" 200 -
```

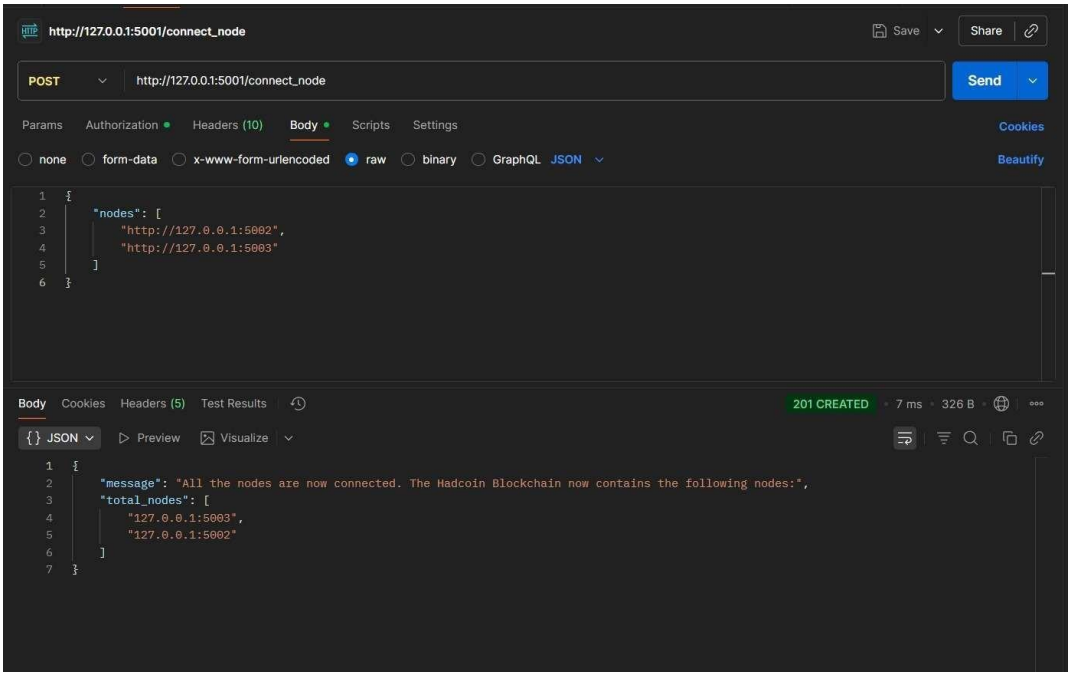
## hadcoin_node_5002

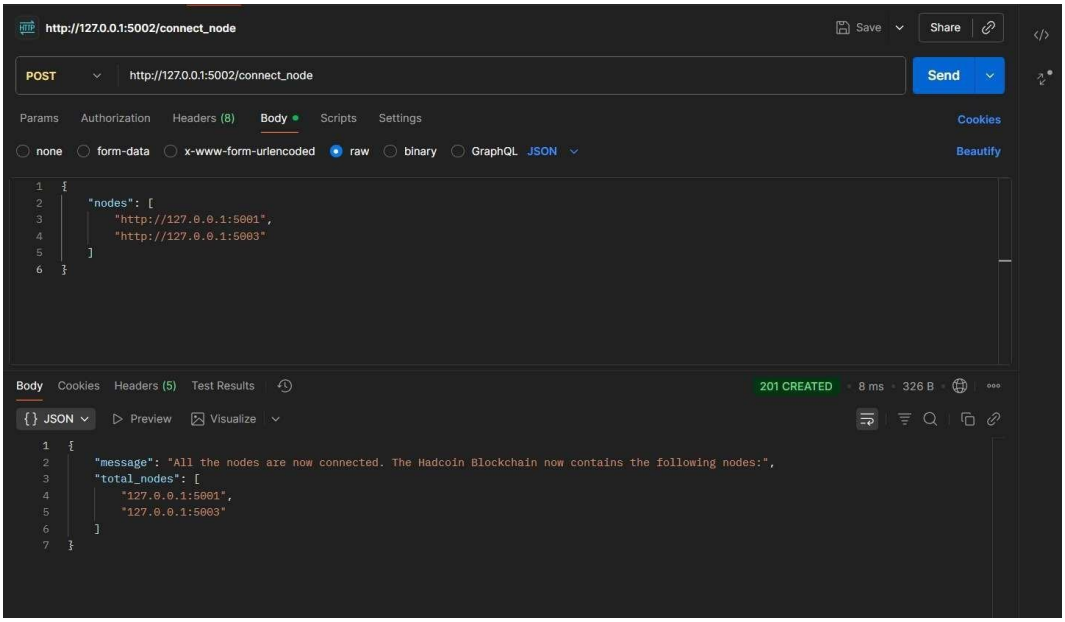
```
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> python hardcoin_node_5002.py
 * Serving Flask app 'hardcoin_node_5002'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5002
 * Running on http://10.124.178.178:5002
Press CTRL+C to quit
127.0.0.1 - - [02/Sep/2026 12:29:43] "POST /connect_node HTTP/1.1" 201 -
127.0.0.1 - - [02/Sep/2026 12:32:10] "POST /connect_node HTTP/1.1" 201 -
127.0.0.1 - - [02/Sep/2026 12:32:52] "POST /connect_node HTTP/1.1" 201 -
127.0.0.1 - - [02/Sep/2026 12:39:20] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [02/Sep/2026 12:50:21] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [02/Sep/2026 12:50:54] "GET /replace_chain HTTP/1.1" 200 -
127.0.0.1 - - [02/Sep/2026 12:51:22] "GET /get_chain HTTP/1.1" 200 -
```

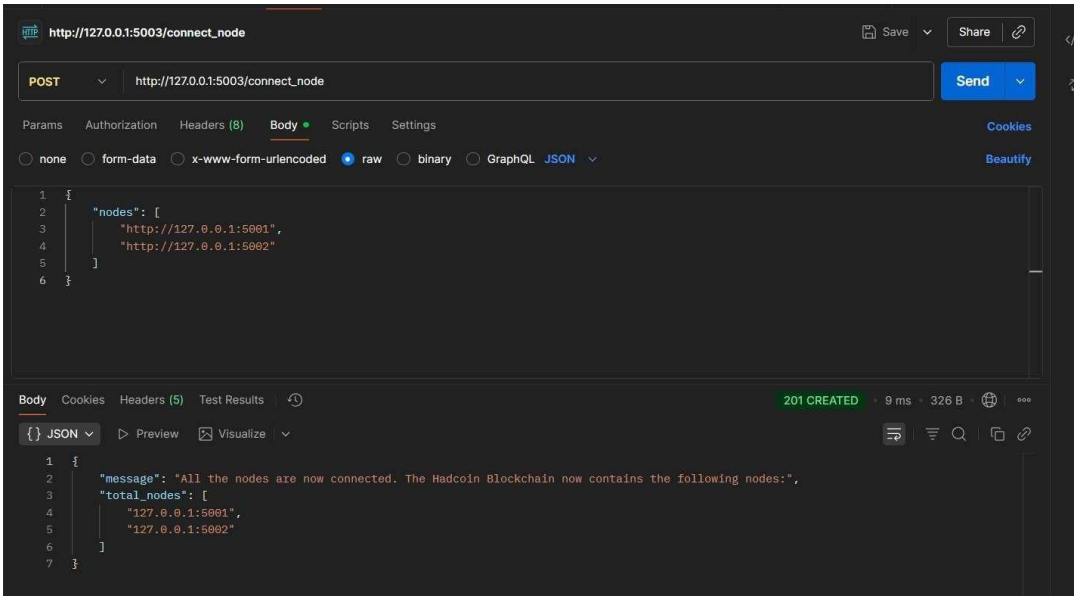
## hadcoin_node_5003

```
PS C:\Clg\Blockchain codes\Exp 03\Experimentss3> python hadoin_node_5003.py
 * Serving Flask app 'hadcoin_node_5003'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5003
 * Running on http://10.124.178.178:5003
Press CTRL+C to quit
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> 127.0.0.1 - - [02/Sep/2026 12:30:17] "GET /connect_node HTTP/1.1" 405 -
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> 127.0.0.1 - - [02/Sep/2026 12:32:10] "GET /connect_node HTTP/1.1" 405 -
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> 127.0.0.1 - - [02/Sep/2026 12:33:55] "GET /connect_node HTTP/1.1" 405 -
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> 127.0.0.1 - - [02/Sep/2026 12:34:20] "GET /connect_node HTTP/1.1" 405 -
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> 127.0.0.1 - - [02/Sep/2026 12:34:22] "POST /connect_node HTTP/1.1" 201 -
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> 127.0.0.1 - - [02/Sep/2026 12:39:59] "GET /get_chain HTTP/1.1" 200 -
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> 127.0.0.1 - - [02/Sep/2026 12:50:26] "GET /get_chain HTTP/1.1" 200 -
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> 127.0.0.1 - - [02/Sep/2026 12:50:54] "GET /get_chain HTTP/1.1" 200 -
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> 127.0.0.1 - - [02/Sep/2026 12:51:22] "GET /replace_chain HTTP/1.1" 200 -
```

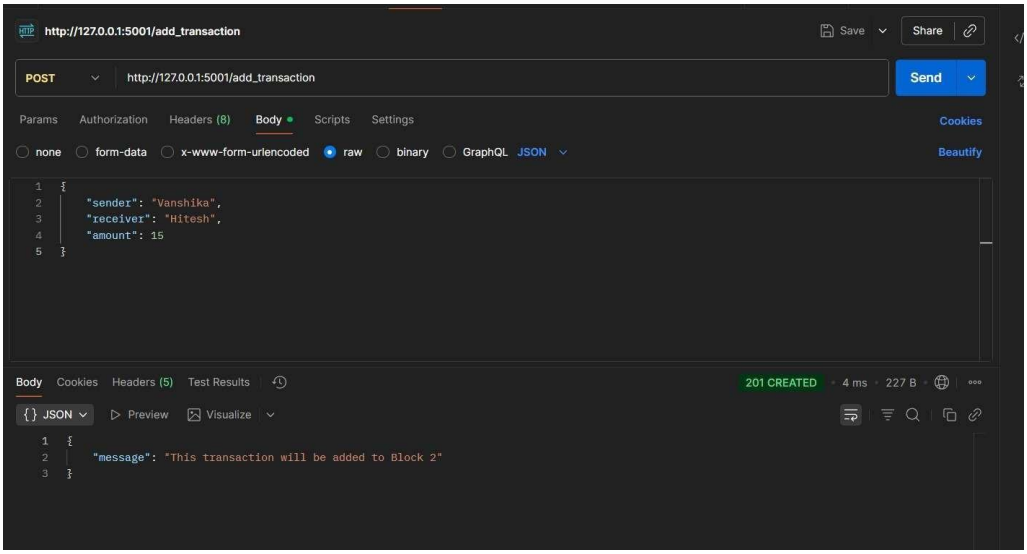
## Connecting the Blockchain Nodes - POST /connect_node.



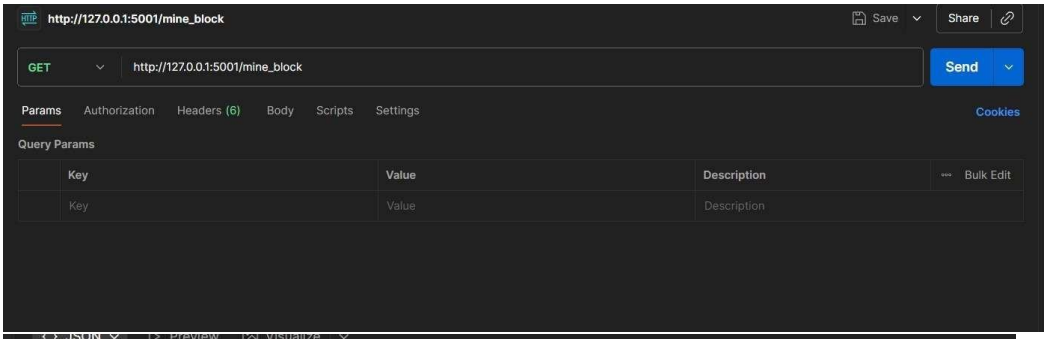

http://127.0.0.1:5002/connect_node
Save
Share
POST
http://127.0.0.1:5002/connect_node
Send
Params
Authorization
Headers (8)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"nodes": [
"http://127.0.0.1:5001",
"http://127.0.0.1:5003"
]
}
Body
Cookies
Headers (5)
Test Results
201 CREATED
8 ms
326 B
{} JSON
Preview
Visualize
{
"message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
"total_nodes": [
"127.0.0.1:5001",
"127.0.0.1:5003"
]
}

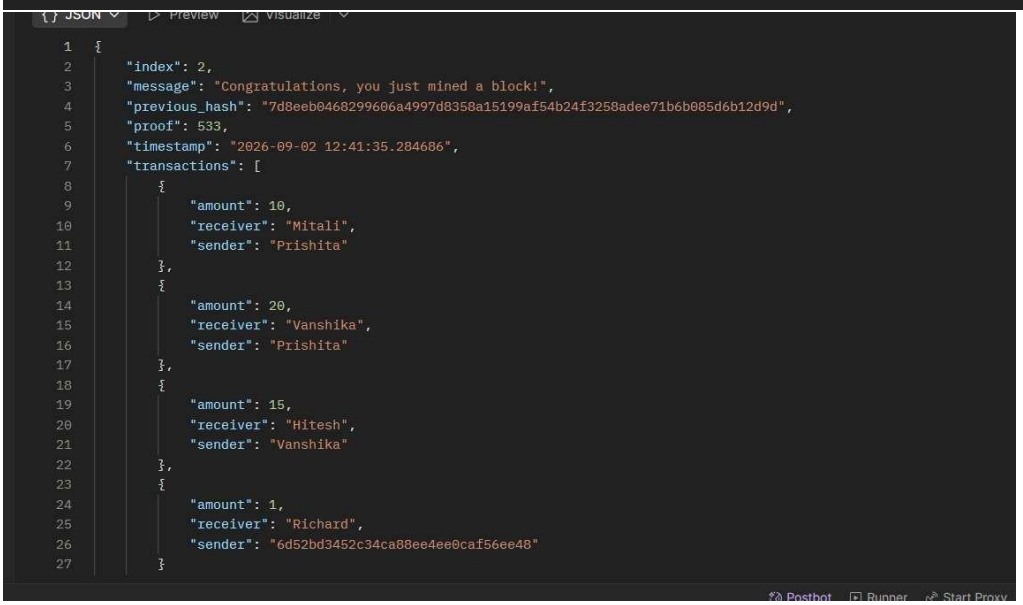

http://127.0.0.1:5003/connect_node
Save
Share
POST
http://127.0.0.1:5003/connect_node
Send
Params
Authorization
Headers (8)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"nodes": [
"http://127.0.0.1:5001",
"http://127.0.0.1:5002"
]
}
Body
Cookies
Headers (5)
Test Results
201 CREATED
9 ms
326 B
{} JSON
Preview
Visualize
{
"message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
"total_nodes": [
"127.0.0.1:5001",
"127.0.0.1:5002"
]
}

## Added Transactions - POST /add_transaction

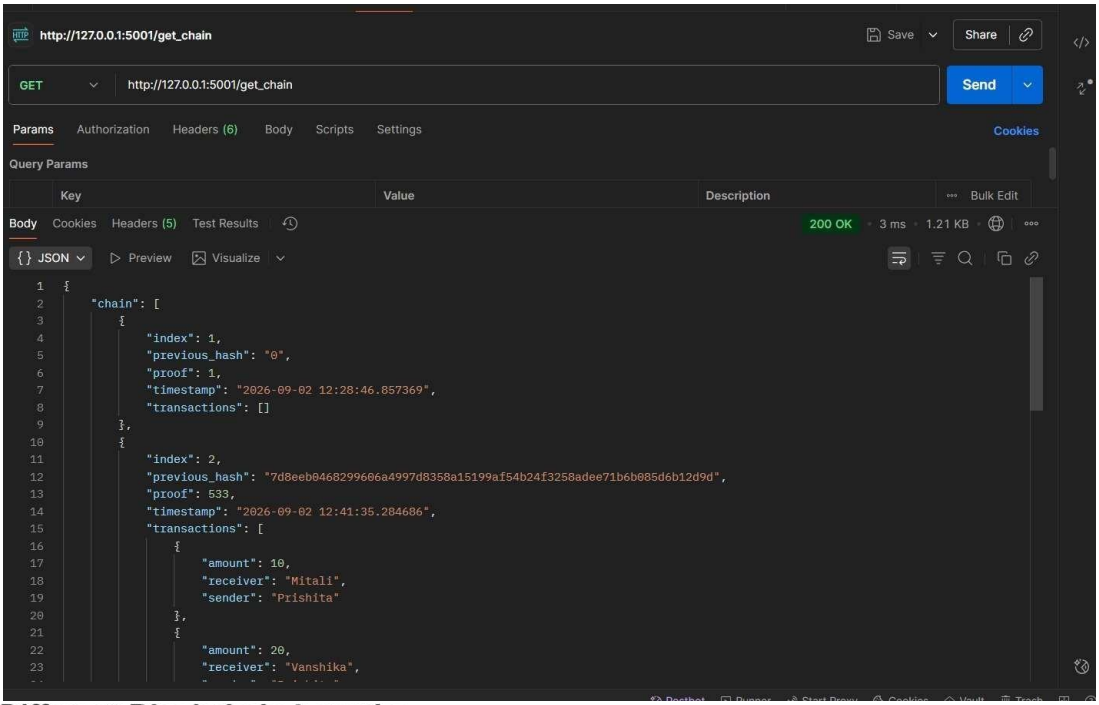



## Mined the Blockchain - GET /mine_block



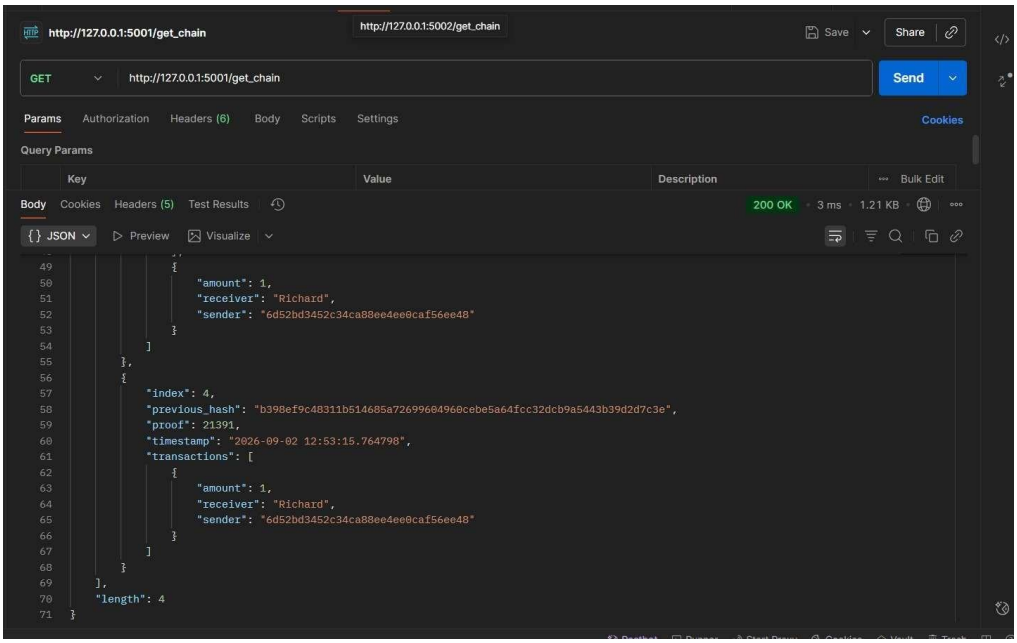



## Fetching the Blockchain - GET /get_chain

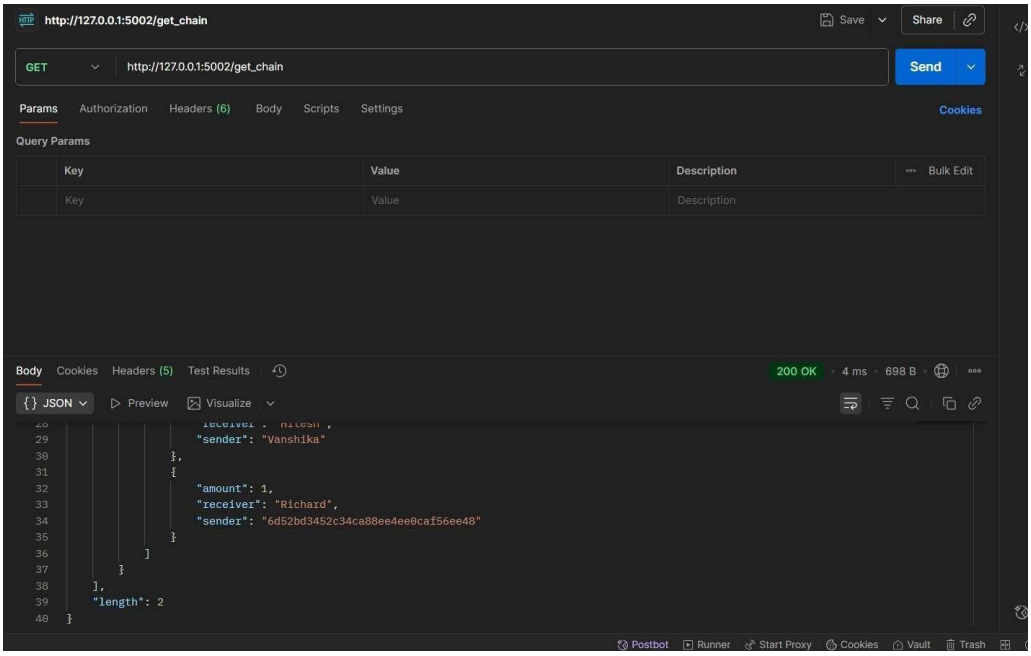



## Different Blockchain Lengths

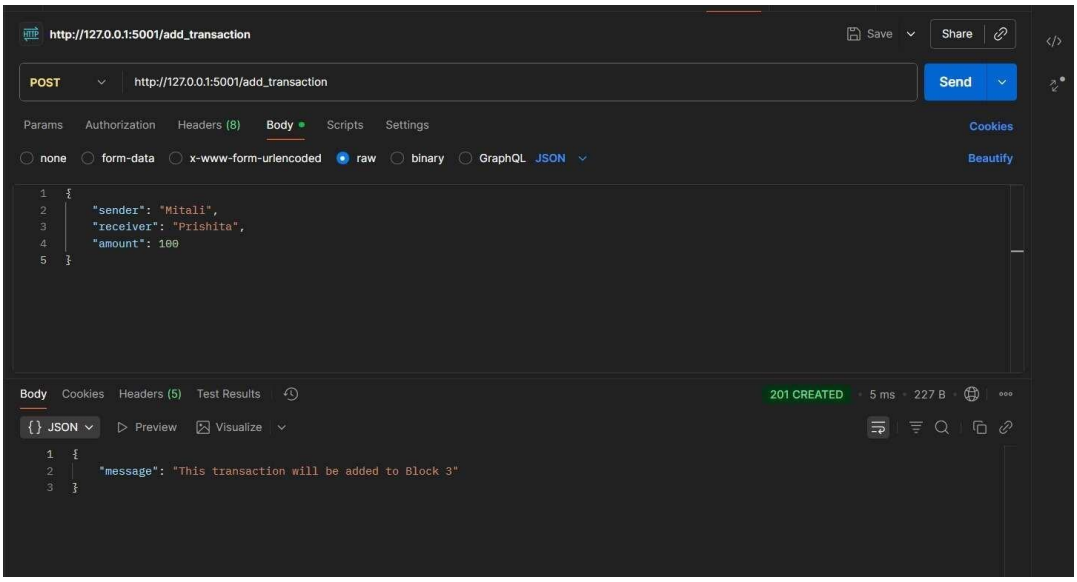
## Chain length of http://127.0.0.1:5001



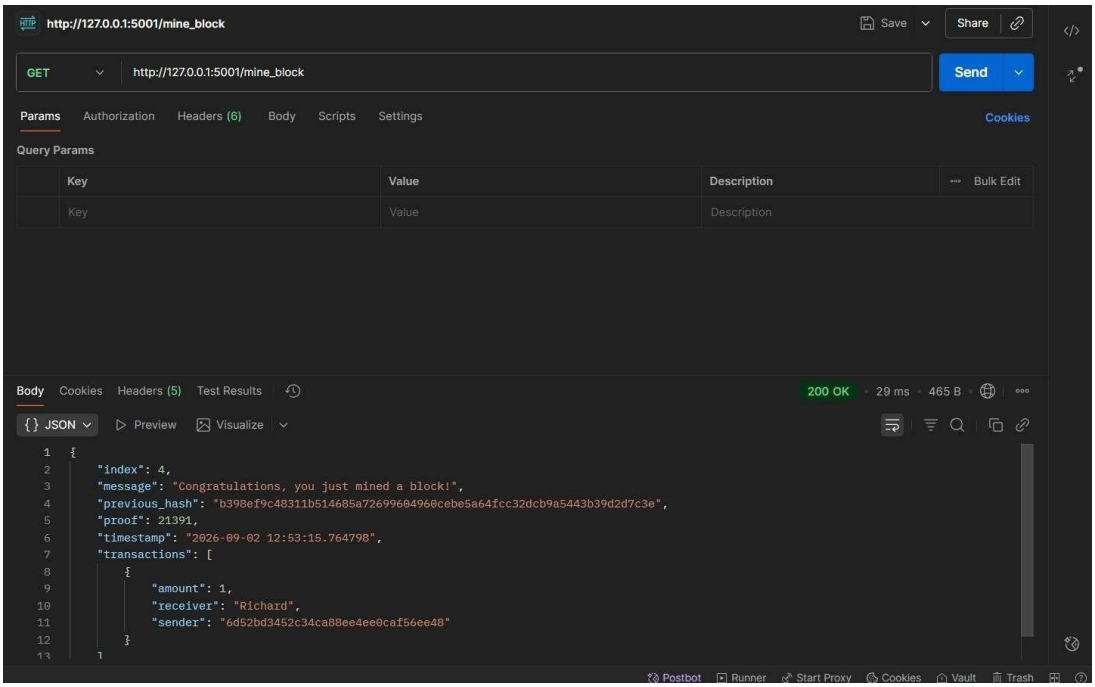

## Chain length of http://127.0.0.1:5002


http://127.0.0.1:5002/get_chain
Save
Share
GET
http://127.0.0.1:5002/get_chain
Send
Params
Authorization
Headers (6)
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Body
Cookies
Headers (5)
Test Results
200 OK
4 ms
698 B
{ } JSON
Preview
Visualize
"sender": "Vanshika"
},
{
"amount": 1,
"receiver": "Richard",
"sender": "6d52bd3452c34ca88ee4ee0caf56ee48"
}
]
}
],
"length": 2
}
Postbot
Runner
Start Proxy
Cookies
Vault
Trash

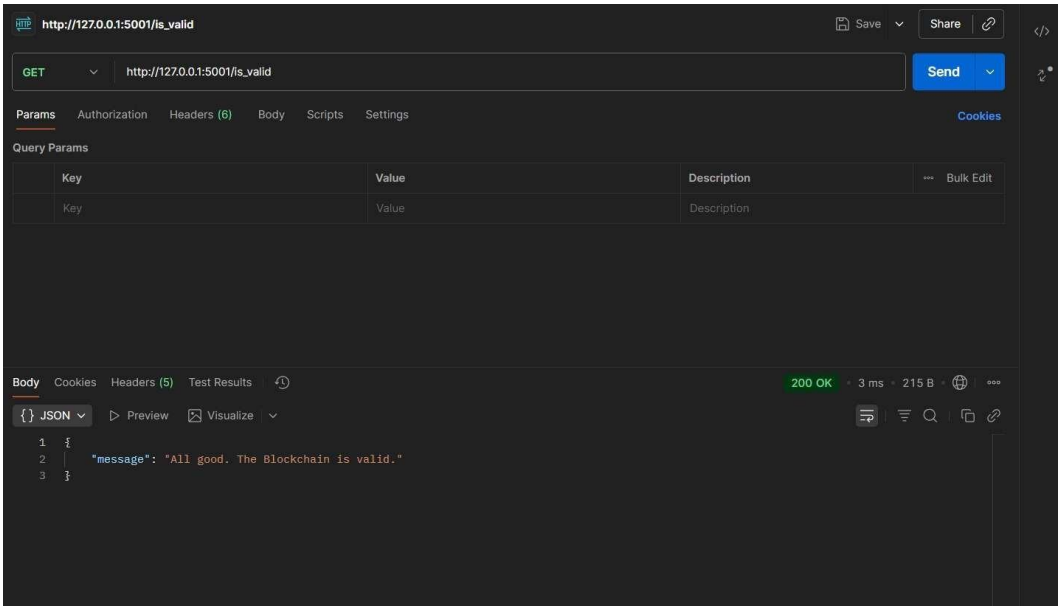
## Add transaction



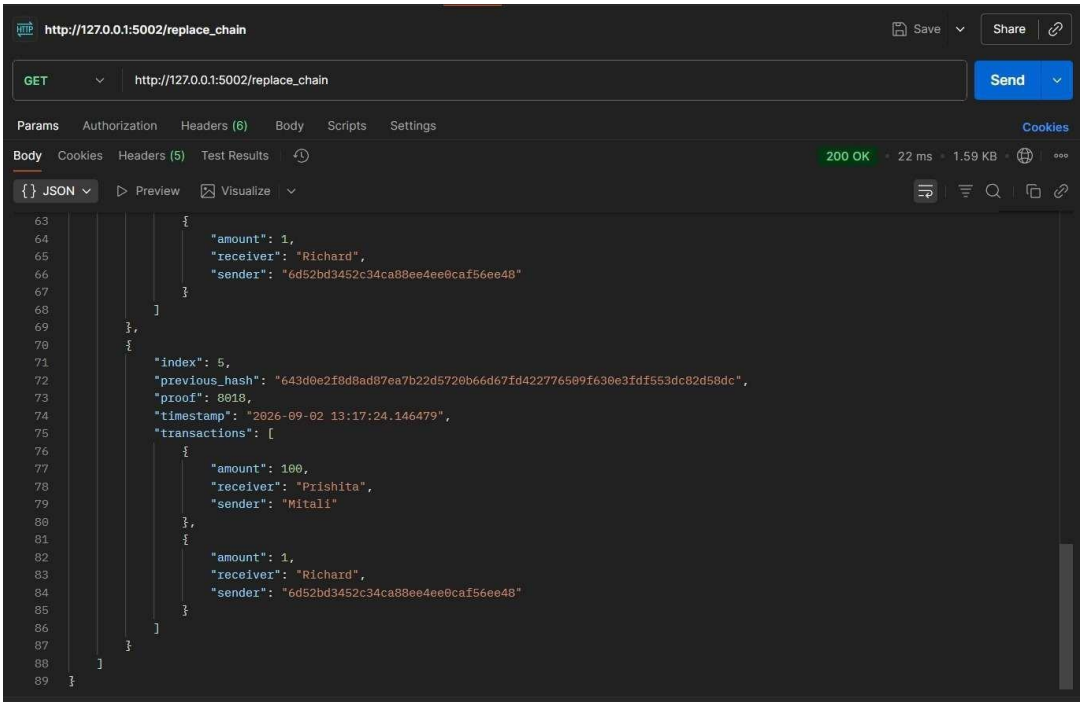

## GET-/mine_block



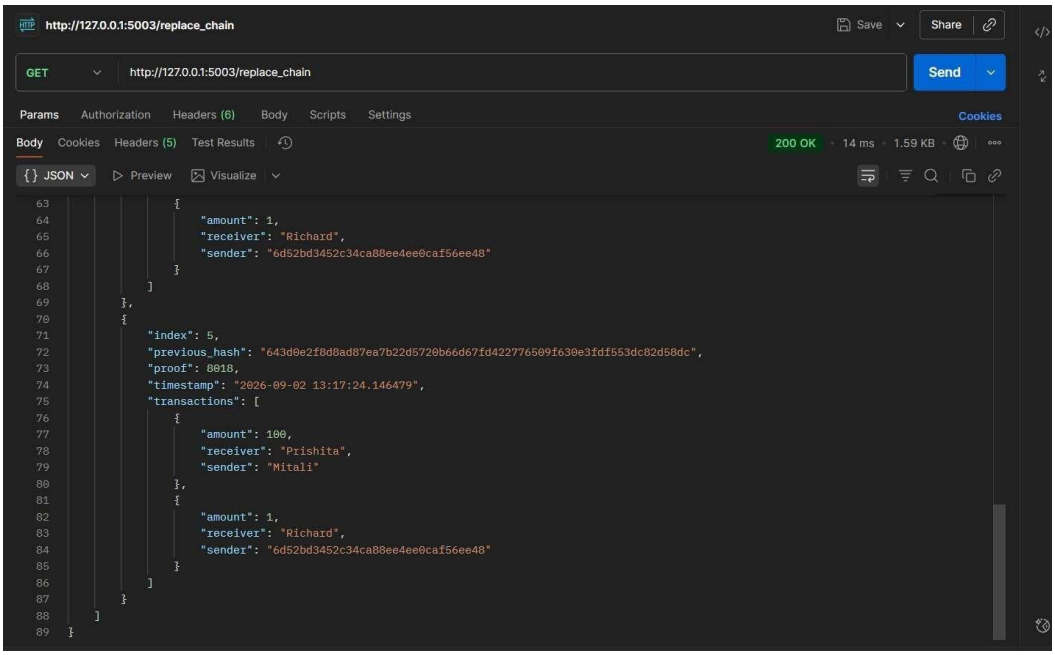
## Validating the Blockchain - GET /is_valid




## Replacing the Chain on Node 5002 - GET /replace_chain

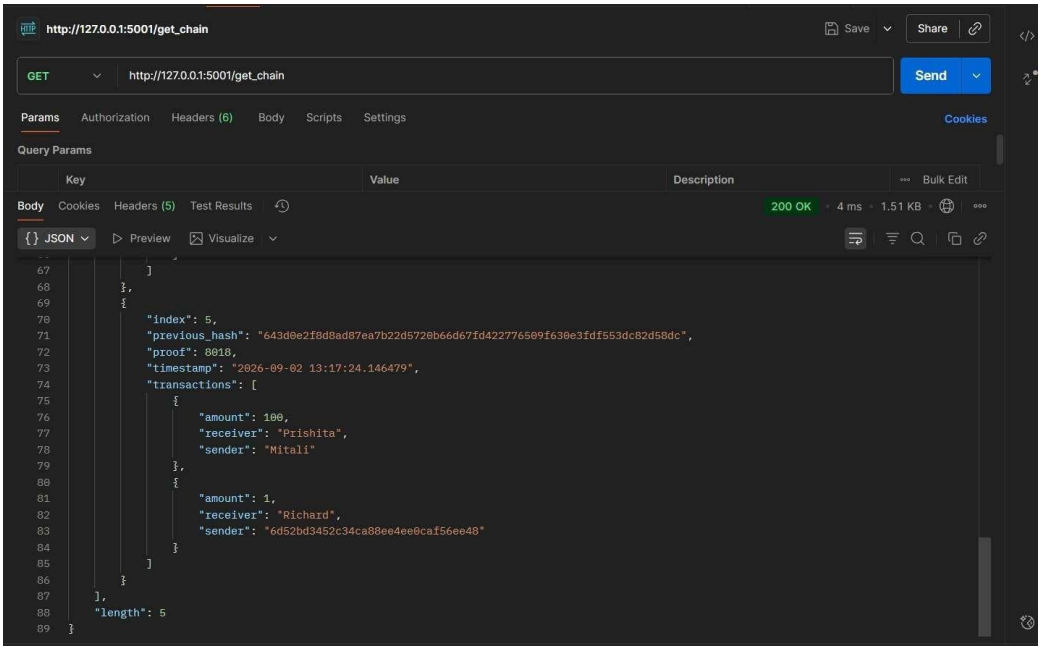


## Replacing the Chain on Node 5003 - GET /replace_chain

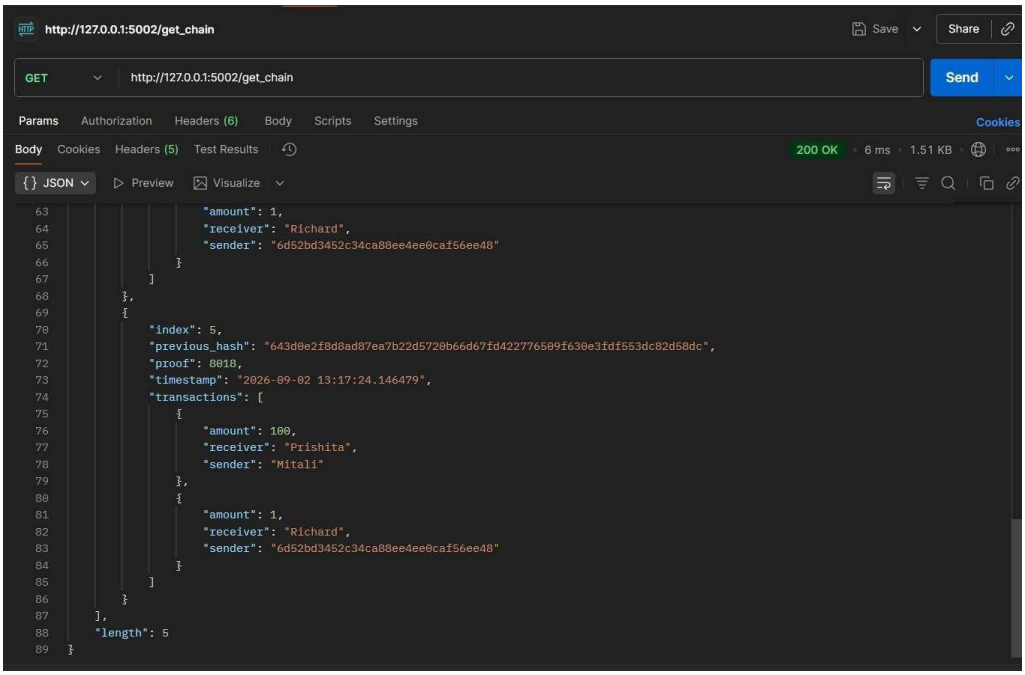



## Final nodes:

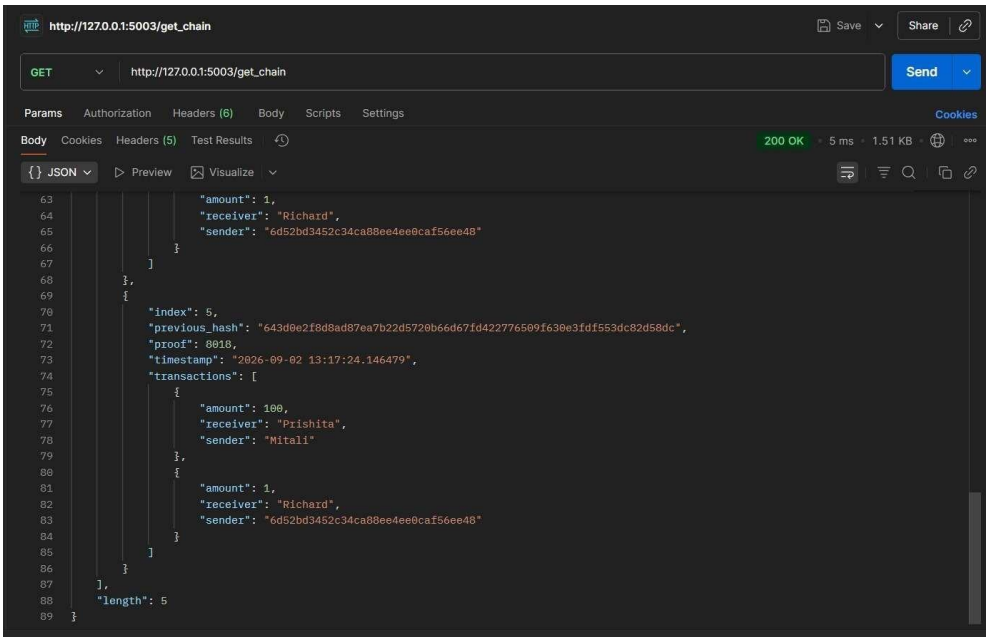
## http://127.0.0.1:5001



**http://127.0.0.1:5002**




**http://127.0.0.1:5003**




## Conclusion:

A cryptocurrency was successfully created using Python and implemented using three blockchain nodes. Transactions were added to the blockchain and blocks were mined using the Proof-of-Work mechanism. The pending transactions were removed after being added to a block. Finally, all nodes were synchronized using the Longest Chain Rule, demonstrating blockchain mining, transactions, validation, and network synchronization.